Comprehensive Viva Preparation Guide

Predictive Weather Route Planner

This document contains highly probable viva questions you may face during your presentation, along with strong, technically convincing answers.

Section 1: Machine Learning vs. APIs (The Dataset Question)

Q1: Why did you use external APIs instead of training your own Machine Learning model on a historical weather dataset? Answer: "Weather forecasting is a highly complex meteorological science. Accurate predictions require processing massive amounts of real-time satellite imagery, atmospheric pressure data, and ocean currents using supercomputers. If we trained a model on historical CSV data, it would only give us an 'average' guess based on the past, which is useless for live, dynamic travel safety. OpenWeather already runs these advanced predictive models. Therefore, our project functions as an **Intelligent Decision-Support System and API Orchestrator**. Our achievement isn't predicting raw weather from scratch; our achievement is programmatically fetching, syncing, and contextualizing that dynamic data against a live pathfinding algorithm."

Section 2: Database Architecture (Relational vs. NoSQL)

Q2: Why did you choose a flat JSON file database instead of a relational database like MySQL or PostgreSQL? Answer: "We evaluated the shape of our data and realized a relational database was unnecessary. A route search is a flat, singular, transactional event (Start, Destination, Time, Weather). There are no complex relational entities—like 'Users' having multiple 'Orders' with multiple 'Products'—that require SQL JOIN operations. Because our data structure natively mirrors JSON, storing it directly as JSON prevents the need for heavy Object-Relational Mappers (ORMs). This serverless, NoSQL Document-Store approach minimizes overhead, eliminates database configuration hurdles, and makes the system incredibly fast and portable."

Q3: Is a file-based JSON database scalable? What if your application grows? Answer: "For the scope of this prototype, a local JSON file is highly performant and handles the I/O load perfectly. However, because we used a decoupled Node.js and Express backend, our data access layer is isolated. If the application needed to scale for millions of users, we could seamlessly swap the JSON file store for a scalable NoSQL cloud database like MongoDB or AWS DynamoDB without rewriting the core logic or touching the frontend."

Section 3: Core Algorithms & Application Logic

Q4: How does your system actually find the "midpoint" of the route? Does it just draw a straight line between the two cities? Answer: "No, it does not draw a straight line. That would give inaccurate results, especially in mountainous or curved roads. When the OSRM routing engine returns the path, it provides a 'polyline'—an array of hundreds of geographic coordinates representing the actual physical road. Our algorithm calculates the cumulative geographical distance of that entire polyline. It then iterates through the coordinates until it

finds the specific coordinate that sits exactly at `Total Distance / 2`. This ensures the midpoint is genuinely on the road halfway through the drive."

Q5: Explain how you synchronize the weather with the route. How do you ensure accuracy? Answer: "This is our 'Timeline Generation' algorithm. If a user drives for 4 hours, they need the weather 4 hours from now, not the current weather. First, we add the estimated driving time to the user's departure time to find the exact Unix timestamp of their arrival at the midpoint and destination. OpenWeather provides a 5-day forecast in 3-hour blocks. We use a JavaScript `reduce()` function to iterate through these blocks, mathematically calculating the absolute difference between the forecast timestamp and our estimated arrival timestamp. The algorithm locks in the forecast block with the smallest time difference."

Q6: Your app makes multiple API calls (Geocoding, Routing, 3x Weather). How do you prevent it from being slow? Answer: "We implemented two major optimizations:

1. **Debouncing:** When the user types a city, we delay the Geocoding API call by 400 milliseconds. If they keep typing, the timer resets. This prevents API spamming and lag.
2. **Asynchronous Parallel Fetching:** Instead of fetching the weather for the start, mid, and end points sequentially (which takes 3x longer), we use JavaScript's `Promise.all()`. This executes all three network requests simultaneously, drastically reducing the loading time."

Section 4: Security & Admin Portal

Q7: Why did you implement your own CAPTCHA system using SVG instead of standard images? How does it prevent bots? Answer: "Standard text-based checks are easily bypassed by bots. We used an SVG-based CAPTCHA because it draws the characters using vector math rather than pixels. Our backend generates random text, but then mathematically rotates, skews, resizes, and applies random colors to each character, while also drawing random noise lines across the image. This completely defeats Optical Character Recognition (OCR) tools used by bots, ensuring only human administrators can access the database."

Q8: How are you securing the administrator passwords in the backend? Answer: "We never store passwords in plain text. When an admin registers, the backend uses the `bcryptjs` library to cryptographically hash the password with a 'salt' (using a work factor/cost of 10). When logging in, the `bcrypt.compareSync()` function hashes the inputted password and compares it to the stored hash. Even if someone gained access to our `.users.json` file, they would only see mathematically irreversible hashes, keeping the accounts completely secure."

Section 5: General & Objective

Q9: What is the difference between OpenStreetMap (OSM) and Leaflet.js in your project? Answer: "OpenStreetMap is the underlying database of roads and geographical data. Leaflet.js is the JavaScript library we use to actually render that data visually in the browser as an interactive, draggable map."

Q10: What makes your project different from Google Maps? Answer: "Google Maps prioritizes the fastest or shortest route. While it may show you weather at your destination if you search for it separately, it does not automatically synchronize weather forecasts along your timeline of travel. Our project specifically models the future weather you will drive into

and proactively warns you of hazards like thunderstorms or high winds before you begin your journey."

Section 6: Frontend & UI Dynamics

Q11: What is the purpose of "Debouncing" in your search bar, and exactly how is it implemented? Answer: "Debouncing is a performance optimization technique. If a user types 'Kochi' quickly, they hit 5 keys. Without debouncing, the app would make 5 separate API requests to the Nominatim geocoding server in half a second, which could lead to our IP being blocked for spamming. Our debounce function uses `setTimeout` to wait 400 milliseconds after the last keystroke before firing the API request. If the user types another key before the 400ms is up, `clearTimeout` resets the timer."

Q12: Why did you use Vanilla JavaScript instead of a modern framework like React, Vue, or Angular? Answer: "We chose Vanilla JavaScript to demonstrate a strong command of fundamental DOM manipulation, asynchronous programming, and event handling. Frameworks like React are incredibly powerful for large-scale, state-heavy enterprise applications, but they introduce a lot of overhead and dependency bulk. By using Vanilla JS, our application loads almost instantly, relies on zero heavy frontend dependencies, and is much easier to deploy."

Q13: How does the application handle situations where a user enters a location that doesn't exist? Answer: "We implemented robust error handling using `try...catch` blocks. If the Nominatim API returns an empty array (meaning the location wasn't found), our `geocode()` function explicitly throws an Error: `Location not found`. The catch block catches this error, prints it to the console, and triggers a browser `alert()` to notify the user smoothly without crashing the rest of the application."

Section 7: Deep Dive into Routing & APIs

Q14: What is the difference between Forward Geocoding and Reverse Geocoding, and where do you use them? Answer: "Forward Geocoding converts human-readable text (like 'Thrissur') into geographic coordinates (Latitude: 10.52, Longitude: 76.21). We use this when the user types in their start and destination. Reverse Geocoding does the exact opposite—it takes coordinates and finds the nearest address. We use this specifically for the Midpoint. Since we calculate the midpoint as raw math (a coordinate halfway down the physical road), we have to Reverse Geocode that raw coordinate to find the name of the town so we can display it nicely to the user."

Q15: How are the safety warnings generated? Are they hardcoded for specific cities? Answer: "The warnings are entirely dynamic and generated programmatically based on live data. When OpenWeather returns a forecast, it includes a specific 'Weather Condition Code' (an integer). For example, codes between 200 and 299 universally represent Thunderstorms. Codes between 700 and 799 represent atmospheric conditions like Fog or Haze. Our `generateWarnings()` function scans these codes and the wind speed. If it detects a dangerous code or a wind speed over 10 m/s, it dynamically pushes a warning into a JavaScript `Set` (to prevent duplicate warnings for the same condition) and injects it into the UI."

Section 8: Backend Architecture

Q16: You mentioned your system is an 'API Orchestrator'. Can you define what API Orchestration means in the context of your code? Answer: "API Orchestration is the process of integrating multiple separate APIs to work together to achieve a single workflow. In our app, the output of one API becomes the input for the next. The Nominatim API provides the coordinates needed for the OSRM Routing API. The OSRM Routing API provides the duration needed to calculate the future timestamp. That timestamp and the coordinates are then fed into the OpenWeather API. Our JavaScript acts as the 'Orchestrator' conducting this sequence automatically."

Q17: What is CORS, and why is `app.use(cors())` included in your Express backend? Answer: "CORS stands for Cross-Origin Resource Sharing. It is a security feature built into modern web browsers that prevents a frontend running on one port or domain from making HTTP requests to a backend running on a different port (like `localhost:3000`). By including the `cors()` middleware in Express, we explicitly tell the backend server to accept and trust incoming HTTP requests from our frontend interface, preventing the browser from blocking the connection."